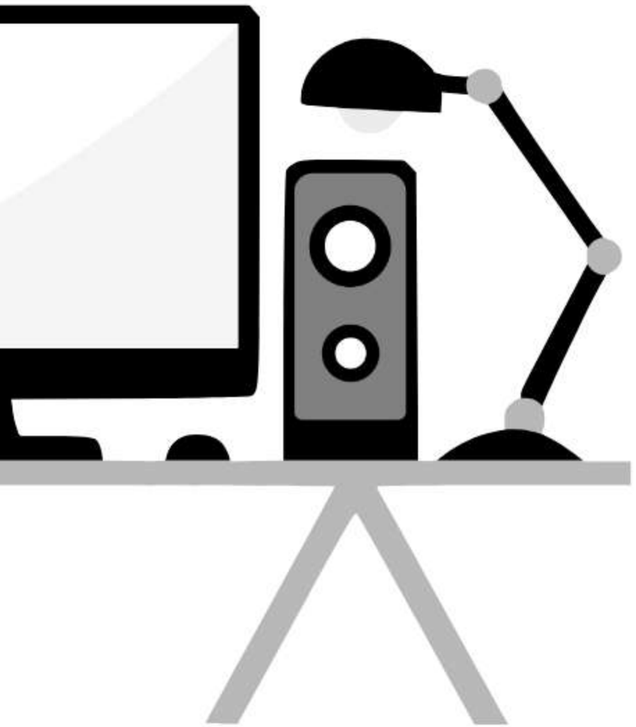
sustantiva
CON•SENTIDO

Energiza!

ALMACENAMIENTO DE DATOS EN VUEX

Contenidos



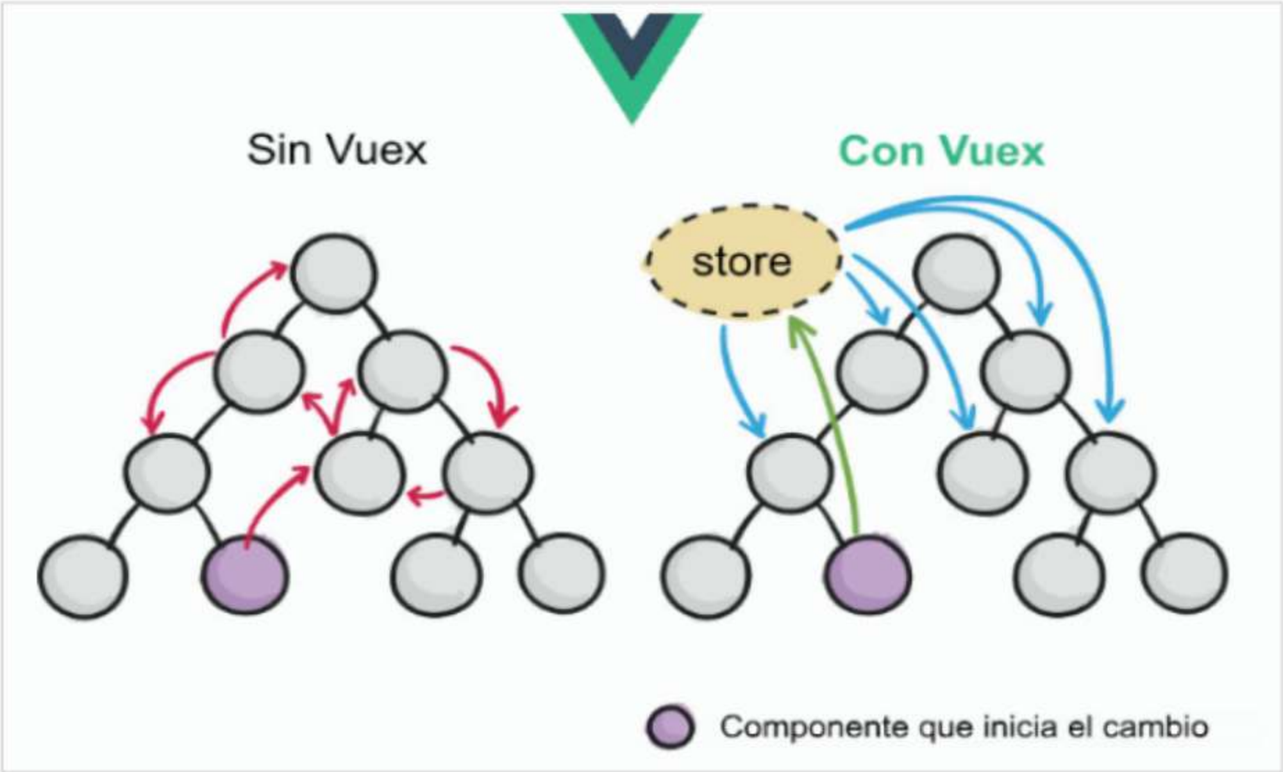
Almacenamiento de estado en Vuex

- Qué es Vuex
- Almacenamiento de estado en Vuex
- Cuándo conviene utilizarlo
- Estructura de la aplicación
- Conceptos claves:
- State
- Mutation
- Action
- Module
- Usos clásicos:
- Manejo de formularios
- Manejo de la autenticación
- Utilizando Vuex y Axios

Qué es Vuex

Vuex es un **patrón y una biblioteca de gestión de estado** para aplicaciones Vue.js. Funciona como un **almacén centralizado (store)** para todos los componentes de la aplicación, con reglas que garantizan que el estado **solo pueda mutarse de forma predecible**.

Como metáfora, podríamos entender a Vuex como un “banco de memoria” del front. Si todos depositan y retiran siguiendo las reglas, no hay balances perdidos.



Almacenamiento de estado en Vuex

Vuex nos provee un lugar centralizado donde almacenar la información compartida en una aplicación Vue: este lugar se llama **estado** o **state**.

Cuando un componente actualiza el state de Vuex, otros componentes también pueden acceder inmediatamente a esa información actualizada.

Sin embargo, permitir cambios al state sin control puede volver el flujo de datos impredecible y difícil de depurar. Para evitar esto, Vuex implementa un **patrón estructurado** que define **cómo leer y modificar el estado**.

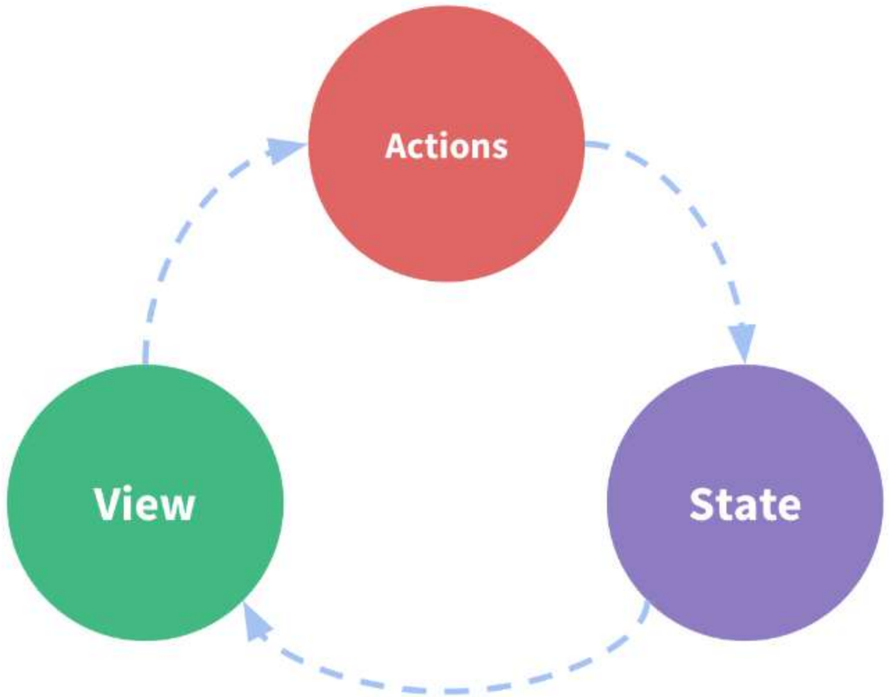
Este patrón se basa en tres elementos clave:

- **Mutations**, que modifican directamente el state
- **Actions**, que gestionan operaciones asincrónicas y luego disparan mutations
- **Getters**, que permiten acceder de forma estructurada a datos derivados del state

Almacenamiento de estado en Vuex

Un **patrón de gestión de estado** describe cómo organizamos los datos que impulsan la interfaz y cómo dichos datos cambian en el tiempo. En una app mínima, como un contador, el **estado** vive en el componente, la **vista** es un mapeo declarativo de ese estado, y las **acciones** son las funciones que modifican el estado en respuesta a eventos del usuario. Ese circuito —estado → vista → acciones → estado— es un **flujo unidireccional** que mantiene todo predecible.

```
const Counter = {
  data: () => ({ count: 0 }),          // estado
  template: `<div>{{ count }}</div>`, // vista
  methods: {                           // acciones
    increment(){ this.count++ }
  }
}
```



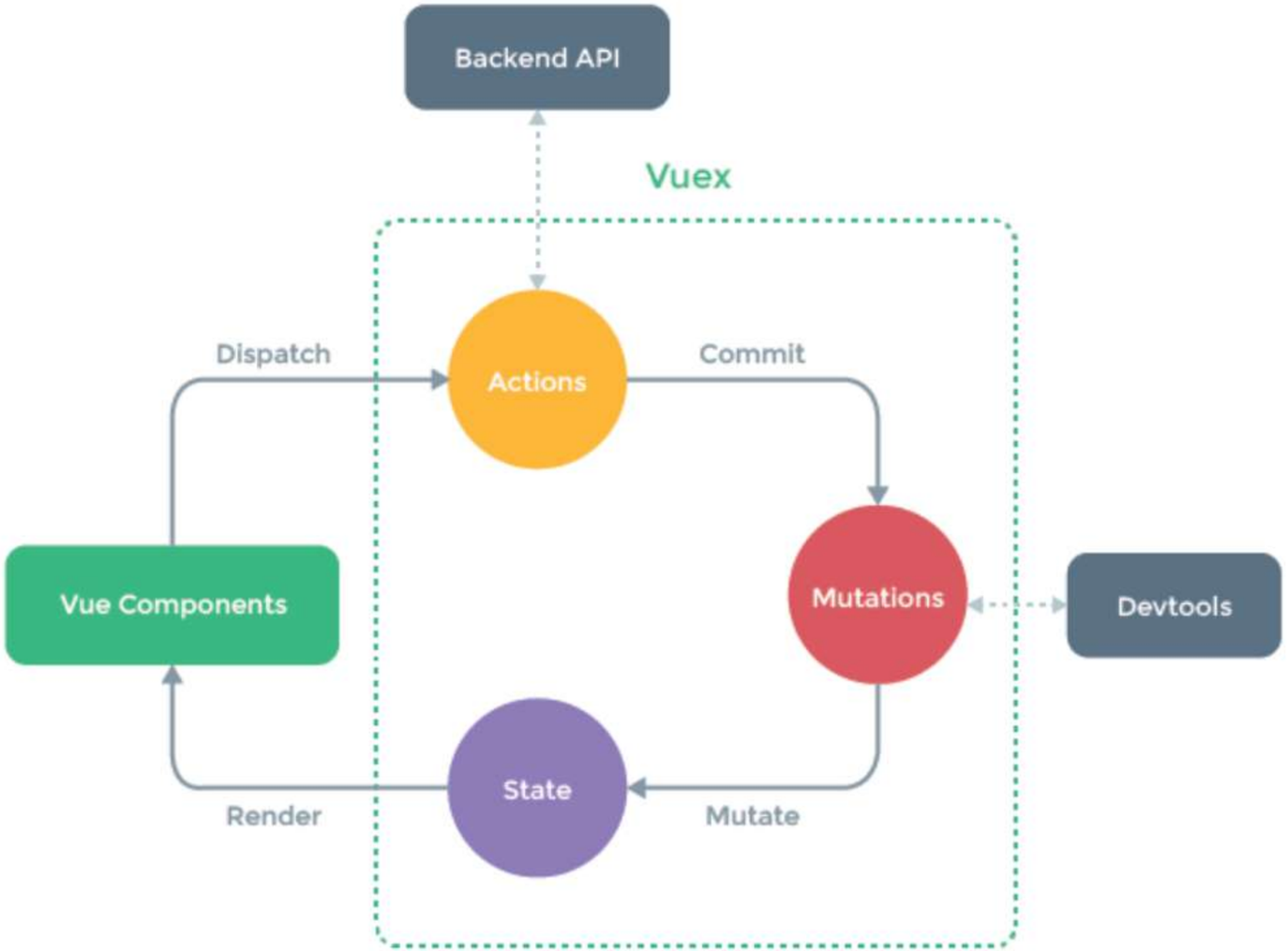
Almacenamiento de estado en Vuex

El problema aparece cuando la aplicación crece. Varias vistas empiezan a depender del **mismo dato** y distintas interacciones quieren **mutar el mismo estado**. Pasar props a través de muchos niveles o coordinar hermanos con eventos y referencias se vuelve frágil: terminamos duplicando información, sincronizándola a mano y peleando con efectos colaterales. Es el “teléfono descompuesto” del front-end.

Almacenamiento de estado en Vuex

La salida es **extraer el estado compartido** desde los componentes y administrarlo en un **store global (un singleton)**. Así, el árbol de componentes pasa a ser una gran "vista" que **lee** ese estado centralizado y **despacha acciones** para solicitar cambios. Al separar con claridad **estado**, **vista** y **acciones**, e imponer reglas para modificar el estado de forma controlada, ganamos **estructura**, **trazabilidad** y **mantenibilidad**.

Esa es la idea detrás de **Vuex**: un patrón y una biblioteca inspirados en Flux, Redux y Elm, pero diseñados específicamente para **Vue.js**, aprovechando su sistema de **reactividad granular** para actualizar la interfaz de manera eficiente y predecible. Menos caos de props y eventos, más orden y flujo claro.



Cuándo conviene utilizarlo

Vuex es especialmente útil en aplicaciones donde **múltiples componentes necesitan compartir o sincronizar datos**, incluso si no tienen una relación directa (por ejemplo, no son padre e hijo).

Si bien Vue ofrece mecanismos como props y events, estos pueden volverse complejos o poco escalables en aplicaciones grandes. **Vuex permite centralizar el estado** y facilitar esta comunicación.

Casos comunes donde conviene usar Vuex:

- **Carrito de compras**: mantener sincronizado el total de productos entre la vista del catálogo y el resumen de compra.
- **Sistema de reservas o agendas**: mostrar en distintos componentes la disponibilidad de horarios.
- **Aplicaciones de monitoreo o dashboards**: múltiples widgets consultando o mostrando los mismos datos en tiempo real.
- **Autenticación de usuario**: mantener disponible el estado de login en toda la app.
- **Apps móviles o SPA complejas**: donde la navegación y el estado están muy interconectados.

Estructura de la aplicación

Vuex no obliga a seguir una estructura específica de carpetas, pero establece algunos **principios fundamentales** que permiten organizar y escalar proyectos de forma efectiva:

- El **estado global** debe estar centralizado en un archivo store.
- Las **mutaciones** son la única forma de cambiar el estado, y deben ser **sincrónicas**.
- Las **acciones** permiten manejar lógica asincrónica (por ejemplo, peticiones a APIs) y luego ejecutar mutaciones.

Para aplicar correctamente estos principios, se recomienda organizar el proyecto con una estructura clara, como la siguiente:

```
├── index.html
├── main.js
├── api/                    # Lógica para peticiones HTTP
│   └── ...
├── components/             # Componentes Vue
│   ├── App.vue
│   └── ...
└── store/                  # Carpeta principal del store Vuex
    ├── index.js           # Ensambla módulos y exporta el store
    ├── actions.js         # Acciones raíz
    ├── mutations.js       # Mutaciones raíz
    └── modules/           # Módulos individuales del store
        ├── cart.js        # Módulo para el carrito
        └── products.js    # Módulo para los productos
```

Conceptos claves

State (estado): Fuente única de la verdad. Son los **datos reactivos** del store que comparten los componentes. No se modifica directamente desde la vista.

Getters: Lecturas **derivadas** del state (como *computed*). Sirven para filtrar, formatear o calcular valores y se cachean según sus dependencias.

Mutations: Única forma **sincrónica** de cambiar el state. Se invocan con commit('mutation', payload) y deben ser **deterministas**.

Actions: Contienen la **lógica de negocio** y la **asincronía** (llamadas a API). Terminan haciendo commit a mutations. Se invocan con dispatch('action', payload).

Modules: Permiten **organizar** el store por dominios y aislar responsabilidades (namespaced: true).

State

En Vuex, el **state** es la *fuente única de la verdad*: aquí viven los datos **reactivos y compartidos** por varios componentes. Para mantener el flujo predecible, **no se modifica desde la vista**; cualquier cambio debe pasar por **mutations**. Con esto evitamos tener "copias" del mismo dato en distintos componentes y los clásicos descalces de información.

Declarar el state en el store
Primero definimos el store con su estado inicial. Observa que state es **una función** que devuelve el objeto con los datos; así mantenemos instancias limpias y reactividad garantizada. Además, declaramos una **mutation** para actualizar el estado de forma controlada.

```
// src/store/index.js  (Vue 3 + Vuex 4)
import { createStore } from 'vuex'

export default createStore({
  state: () => ({
    numero: 10,         // dato compartido simple
    productos: [],      // lista global disponible en toda la app
    usuario: null       // se setea tras el login
  }),
  mutations: {
    // única vía válida para cambiar 'numero'
    setNumero(state, n) { state.numero = n }
  }
})
```

State

Conectar el store a la aplicación

Ahora “enchufamos” el store al árbol de componentes. La clave es **inyectarlo antes de montar** la app; así this.$store (Options) o useStore() (Composition) estarán disponibles en cualquier componente.

Desde este punto, tu aplicación conoce el store y puedes leer o mutar el estado desde las vistas con las APIs de Vuex.

```
// src/main.js
import { createApp } from 'vue'
import App from './App.vue'
import store from './store'

createApp(App).use(store).mount('#app')
```

State

Leer el state (Options API)

Aquí **exponemos el state como propiedades computadas** usando mapState. Esto crea un enlace reactivo: si el store cambia, la vista se actualiza sola sin que tengamos que suscribirnos manualmente.

Fíjate que **no** accedemos a $store.state.numero en el template; en su lugar usamos numero, que ya es una computed conectada al store.

```
<script>
import { mapState } from 'vuex'
export default {
  computed: {
    ...mapState(['numero', 'productos'])
  }
}
</script>

<template>
  <h1>Valor actual: {{ numero }}</h1>
  <list-product :items="productos" />
</template>
```

State

Leer el state (Composition API)

Con <script setup> usamos useStore() para obtener la instancia del store y computed para mantener la reactividad. Es el mismo concepto que en Options API, solo cambia la sintaxis.

De nuevo, **no** leemos el state "a pelo" en el template; lo exponemos como computed para que Vue actualice la UI automáticamente cuando haya cambio

```
<script setup>
import { computed } from 'vue'
import { useStore } from 'vuex'

const store = useStore()
const numero = computed(() =>
store.state.numero)
const productos = computed(() =>
store.state.productos)
</script>

<template>
  <h1>Valor actual: {{ numero }}</h1>
  <list-product :items="productos" />
</template>
```

State

Mostrar una lista desde el store (conexión completa)

Primero, definimos en el state una colección de productos. Esto hace que la lista sea **global y compartida** (no una copia local por componente).

```
// en el state del store
productos: [
  { id: 'p-01', nombre: 'Producto 1', precio: 1200, stock: 20 },
  { id: 'p-02', nombre: 'Producto 2', precio: 1500, stock: 10 }
]
```

State

Luego, en una vista, **leemos** esa lista desde el store y la usamos en el template. La conexión es directa: el array vive en el store; la vista solo lo **refleja**.

Si en el futuro un componente agrega, edita o elimina un producto mediante **mutations**, esta vista se actualizará sin que tengamos que “sincronizar” nada a mano.

```
<script>
import { mapState } from 'vuex'
export default {
  computed: {
    ...mapState(['productos'])
  }
}
</script>

<template>
  <section>
    <h2>Productos ({{ productos.length }})</h2>
    <ul>
      <li v-for="p in productos" :key="p.id">
        {{ p.nombre }} — ${{ p.precio }} (stock: {{ p.stock }})
      </li>
    </ul>
  </section>
</template>
```

State

Cambiar el state correctamente

Para modificar el estado, **comiteamos** una mutation. Esto mantiene el flujo predecible y deja rastro claro en Devtools.

```
// en algún método o action
this.$store.commit('setNumero', 42)      // Options API
// o bien
store.commit('setNumero', 42)            // Composition API con useStore()
```

No uses store.state.numero++. Saltarse las mutations rompe la trazabilidad y hace más difícil depurar.

Qué entra (y qué no) en el state

El state es ideal para **datos compartidos y persistentes** (perfil de usuario, permisos, catálogo, carrito). En cambio, los **estados efímeros de UI** (abrir/cerrar un modal, texto temporal de un input) suelen vivir **localmente** en el componente. Tampoco guardes **valores derivados** como “total de productos”; calcúlalos con un **getter** o con computed en la vista para evitar desincronizaciones.

Mutation

Qué son y reglas básicas

Una **mutation** es la **única** forma de cambiar el state en Vuex y **siempre es sincrónica**. Recibe el state (y opcionalmente un payload) y realiza un cambio **determinista**: con la misma entrada, mismo resultado.

Esta disciplina mantiene el flujo predecible y permite **time-travel** en Devtools.

```
// store/modules/agenda.js
export default {
  namespaced: true,
  state: () => ({ ocupado: false }),
  mutations: {
    // Cambia un booleano del state de forma controlada
    toggleOcupado(state) {
      state.ocupado = !state.ocupado
    },
    // Variante explícita: fija el valor desde fuera
    setOcupado(state, valor) {
      state.ocupado = Boolean(valor)
    }
  }
}
```

Cómo se usa desde un componente (commit):

```
// Options API
this.$store.commit('agenda/toggleOcupado')

// Composition API
import { useStore } from 'vuex'
const store = useStore()
store.commit('agenda/setOcupado', true)
```

Mutation

Mutations con payload (parámetros)

Cuando el cambio depende de datos externos, pasa un **payload** claro y mínimo. Evita “cargar” la mutation con lógica de negocio compleja; eso vive en **actions**.

Por qué así y no con .map(...): usar map para mutar elementos es confuso (parece que creas un array nuevo, pero estás mutando dentro). Con find/splice dejas claro **qué** elemento cambia y **cómo**, y mantienes la reactividad (en Vue 3 no necesitas Vue.set).

```
// store/modules/catalogo.js
export default {
  namespaced: true,
  state: () => ({
    productos: [
      { id: 'p-01', nombre: 'Producto 1', stock: 20 },
      { id: 'p-02', nombre: 'Producto 2', stock: 10 }
    ]
  }),
  mutations: {
    // Ajusta el stock del producto indicado
    actualizarStock(state, { id, delta }) {
      const p = state.productos.find(p => p.id === id)
      if (!p) return
      p.stock += delta
    },
    // Reemplaza un producto completo de forma reactiva
    reemplazarProducto(state, productoActualizado) {
      const i = state.productos.findIndex(p => p.id ===
productoActualizado.id)
      if (i !== -1) state.productos.splice(i, 1, productoActualizado)
    }
  }
}
```

Mutation

Dónde va la asincronía

Nunca pongas fetch/axios/await dentro de una **mutation**. La asincronía y la lógica de negocio van en **actions**; cuando obtienes el resultado, **comiteas** la mutation.

```
// Bien: la API en actions, la escritura en mutations
import api from '@/api/api'

export default {
  namespaced: true,
  actions: {
    async agregarProducto({ commit }, nuevo) {
      const { data } = await api.post('/productos', nuevo)
      commit('reemplazarProducto', data)
    }
  }
}
```

Action

Una **Action** concentra la **lógica de negocio** y la **asincronía** (fetch/axios). No cambia el state directamente: **después** del trabajo, hace commit a una **mutation**.

Flujo: *Action (async) → commit → Mutation (sync) → State → View*.

```
// módulo agenda (namespaced)
export default {
  namespaced: true,
  state: () => ({ ocupado: false }),
  mutations: {
    toggleOcupado(s){ s.ocupado = !s.ocupado }
  },
  actions: {
    toggleOcupado({ commit }){ commit('toggleOcupado') }
  }
}
```

Usamos una acción para encapsular el gesto y mantener el flujo abierto a crecer (mañana puede ser async sin tocar la vista).

La vista **despacha** la action; la **mutation** es quien cambia el state.

Action

Action asíncrona con Axios

La asincronía vive en la action; el cambio de estado, en la mutation.

Primero obtenemos datos, **luego** comiteamos; nunca await dentro de mutations.

```
// módulo catalogo
import api from '@/api/api'
export default {
  namespaced: true,
  state: () => ({ productos: [] }),
  mutations: {
    setProductos(s, lista){ s.productos = lista }
  },
  actions: {
    async cargarProductos({ commit }){
      const { data } = await api.get('/productos')
      commit('setProductos', data)
    }
  }
}
```

Action

Despachar desde la vista

El componente no conoce endpoints ni mutations: solo **despacha**.

```
<!-- Options API -->
<script>
import { mapActions } from 'vuex'
export default {
  methods: { ...mapActions('catalogo', ['cargarProductos']) },
  mounted(){ this.cargarProductos() }
}
</script>

<!-- Composition API -->
<script setup>
import { onMounted } from 'vue'
import { useStore } from 'vuex'
const store = useStore()
onMounted(() => store.dispatch('catalogo/cargarProductos'))
</script>
```

Module

¿Por qué módulos?

Un único árbol de estado crece rápido y se vuelve difícil de mantener. **Módulos** permiten **partir el store por dominios** (auth, catálogo, agenda…), cada uno con su **state, mutations, actions y getters**. Así aislas responsabilidades y mantienes el código legible.

```
// Definición de dos módulos
const moduleA = {
  namespaced: true,
  state: () => ({ count: 0 }),
  mutations: { increment(s){ s.count++ } }
}

const moduleB = {
  namespaced: true,
  state: () => ({ items: [] }),
  mutations: { setItems(s, xs){ s.items = xs } }
}

// Registro en el store
import { createStore } from 'vuex'
export default createStore({
  modules: { a: moduleA, b: moduleB }
})

// Acceso
// store.state.a.count   → state local de A
// store.state.b.items   → state local de B
```

Module

Estado local vs raíz

Dentro de **mutations y getters**, state es **el estado local del módulo**.

En **actions**, además de commit/dispatch, tienes **state (local)** y **rootState (raíz)**. Los **getters** reciben (state, getters, rootState, rootGetters).

```
const moduleA = {
  namespaced: true,
  state: () => ({ count: 0 }),
  mutations: {
    increment(state){ state.count++ }            // state = local
  },
  getters: {
    doubleCount(state){ return state.count * 2 } // state = local
  },
  actions: {
    incrementIfOddOnRootSum({ state, commit, rootState }) {
      // Usa local + raíz para decidir
      if ( (state.count + rootState.total) % 2 === 1 )
         commit('increment')
    }
  }
}
```

Module

Namespaces desde la vista (y entre módulos)

Con namespaced: true, las vistas trabajan **por espacio de nombres**. Usa mapState / mapActions apuntando al módulo. Entre módulos, puedes **dispatch/commit** al raíz con { root:true }.

```
<!-- Options API -->
<script>
import { mapState, mapActions } from 'vuex'
export default {
  computed: { ...mapState('a', ['count']) },
  methods:   { ...mapActions('a',
['incrementIfOddOnRootSum']) }
}
</script>

<!-- Composition API -->
<script setup>
import { computed } from 'vue'
import { useStore } from 'vuex'
const store = useStore()
const items = computed(() => store.state.b.items)
</script>
```

```
// Desde un action de A, llamar algo en 'auth' (otro módulo)
dispatch('auth/refreshToken', null, { root: true })
// O comitear en otro módulo
commit('b/setItems', nuevaLista, { root: true })
```

Module

Puedes **anidar** módulos (útil en áreas grandes) y **registrarlos dinámicamente** (lazy load por ruta).

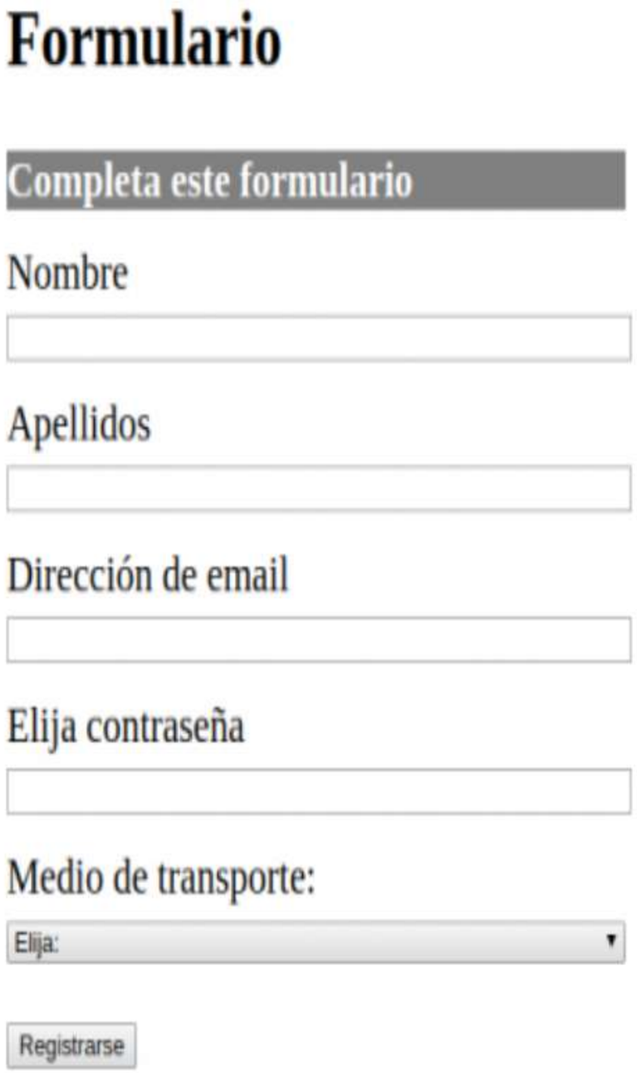
```
// Anidado
const admin = {
  namespaced: true,
  modules: {
    users: { namespaced: true, state: () => ({ list: [] }) }
  }
}

// Dinámico
store.registerModule('cart', { namespaced:true, state:()=>({ items:[] }) })
// ...luego
store.unregisterModule('cart')
```

Usos clásicos: Manejo de formularios

Manejo de formularios con Vue/Vuex

Los formularios son un caso típico de gestión de estado. Cuando el dato **se usa en varias vistas** o **debe persistir entre pasos/rutas** (registro, wizard, edición prolongada), conviene apoyarse en Vuex. Si es un formulario simple y local, mantenemos el modelo en el componente y solo **sincronizamos** con el store al guardar.

Este formulario recoge **Nombre, Apellidos, Email, Contraseña** y un **Medio de transporte**.



Usos clásicos: Manejo de formularios

Requisitos del formulario

La entrada del usuario se **sincroniza con la vista** (v-model) y el envío valida los campos antes de continuar. Si hay problemas, se muestra un **panel de errores** en rojo para que el usuario pueda corregir sin perder lo ya escrito.

Para que el formulario sea válido, se comprueba que:

- **Nombre** y **Apellidos** no estén vacíos.
- **Email** tenga **formato válido**.
- **Contraseña** cumpla la **longitud mínima** definida.
- **Medio de transporte** esté **seleccionado**.

Usos clásicos: Manejo de formularios

Campo condicional: “Matrícula”

El campo **Matrícula solo aparece** cuando el usuario elige **“Automóvil”** como medio de transporte.

- La visibilidad es **condicional** (se oculta/ muestra sin recargar la página).

- Cuando está visible, **también es obligatorio** y se valida con su propio mensaje de error (“Matrícula requerida”).
  Este patrón mantiene el formulario **limpio**: solo se pide lo que es relevante para la elección del usuario.

Usos clásicos: Manejo de formularios

Envío y feedback al usuario

Al presionar **Registrar**, si todas las validaciones pasan:

1. Se **envían los datos** (por ejemplo, mediante una **Action** que llama a la API y luego actualiza el store).

2. La interfaz muestra un **mensaje de éxito** en verde.

Usos clásicos: Manejo de la autenticación

En la mayoría de proyectos debemos **proteger rutas** y saber si el usuario **está autenticado**. Centralizamos la sesión en un **módulo auth** de Vuex: definimos el estado, cómo cambia (mutations) y cómo se ejecuta el flujo (actions).

En el código mostrado, el login activa auth y guarda el username; el logout limpia ambos.

```
// store/modules/auth.js
export default {
  namespaced: true,
  state: () => ({
    auth: false,
    username: null,
  }),
  mutations: {
    doLogin(state, username) {
      state.auth = true
      state.username = username
    },
    doLogout(state) {
      state.auth = false
      state.username = null
    },
  },
  actions: {
    async doLogin({ commit }, username) {

      commit('doLogin', username)
    },
    doLogout({ commit }) {
      commit('doLogout')
    },
  },
}
```

Usos clásicos: Manejo de la autenticación

Vistas: Dashboard (privada) y Login (pública)

Dashboard es un área privada: muestra el usuario y permite **cerrar sesión**.

Login envía credenciales; si el login resulta, **redirige** al Dashboard. Las vistas **despachan** acciones del store; no modifican el estado directo.

```vue
<!-- UserDashboard.vue -->
<script>
export default {
  methods: {
    async onLogout() {
      await this.$store.dispatch('auth/doLogout')
      this.$router.push({ name: 'Login' })
    },
  },
}
</script>
<template>
  <h1>Dashboard</h1>
  <p>Hola, {{ $store.state.auth.username }}</p>
  <button @click="onLogout">Salir</button>
</template>
```

```vue
<!-- AppLogin.vue -->
<script>
export default {
  data: () => ({ username: '', password: '' }),
  methods: {
    async onSubmit() {
      try {
        await this.$store.dispatch('auth/doLogin', this.username)
        this.$router.push({ name: 'Dashboard' })
      } catch (error) { console.error(error) }
    },
  },
}
</script>
<template>
  <form @submit.prevent="onSubmit">
    <input v-model="username" placeholder="Usuario" />
    <input v-model="password" type="password" placeholder="Contraseña" />
    <button>Entrar</button>
  </form>
</template>
```

Usos clásicos: Manejo de la autenticación

Rutas protegidas + menú

Marcamos las rutas privadas con meta.requiresAuth. El enlace a **Dashboard** funciona solo si hay sesión; si no, el **guard** redirige a **Login**.

```
<!-- Menú -->
<router-link :to="{ name: 'Dashboard' }">Dashboard</router-link>
```

```
// router/index.js
const routes = [
  { path: '/login', name: 'Login', component: () => import('@/views/AppLogin.vue') },
  { path: '/dashboard', name: 'Dashboard',
    component: () => import('@/views/UserDashboard.vue'),
    meta: { requiresAuth: true }
  },
]
```

Usos clásicos: Manejo de la autenticación

Guard global (antes de entrar a una ruta)

El **beforeEach** revisa si la ruta exige autenticación y si el usuario está logueado en el store. Si no lo está, lo envía a **Login**. Con esto se protege toda la zona privada.

```
// router/index.js
import store from '@/store'

router.beforeEach((to, from, next) => {
  const requiere = to.matched.some(r => r.meta.requiresAuth)
  const logueado = store.state.auth.auth
  if (requiere && !logueado) next({ name: 'Login' })
  else next()
})
```

Usos clásicos: Utilizando Vuex y Axios

Utilizando Vuex y Axios

Las apps web leen y actualizan datos desde una **API**. **Axios** hace las peticiones HTTP y **Vuex** centraliza el resultado (lista, loading, error) para compartirlo entre vistas. Así evitamos duplicar lógica y mantenemos un flujo claro:

Action (async) → commit → Mutation (sync) → State → View.

Instalación:

```
# npm
npm i axios
# yarn
yarn add axios
```

Usos clásicos: Utilizando Vuex y Axios

Una sola instancia de Axios (configurada)

Creamos una instancia con baseURL (tu json-server en localhost:3000). Más adelante puedes sumar headers de auth sin tocar los componentes.

```
// src/api/api.js
import axios from 'axios'

const api = axios.create({
  baseURL: 'http://localhost:3000',
  timeout: 8000
})

export default api
```

Con esto evitamos “import axios y pegar URL” en cada componente.

Usos clásicos: Utilizando Vuex y Axios

Vuex + Axios: cargar datos desde la API

El módulo guarda la **lista** y los flags de **carga/errores**. La **action** llama a la API y **luego** comitea la mutation (nunca await dentro de mutations).

```
// src/store/modules/frameworks.js
import api from '@/api/api'

export default {
  namespaced: true,
  state: () => ({ items: [], loading: false, error: '' }),
  mutations: {
    setLoading(s,v){ s.loading = v },
    setError(s,e){ s.error = e || '' },
    setItems(s,lista){ s.items = lista }
  },
  actions: {
    async cargar({ commit }) {
      commit('setLoading', true); commit('setError','')
      try {
        const { data } = await api.get('/frameworks')   // GET
http://localhost:3000/frameworks
        commit('setItems', data)
      } catch (e) {
        commit('setError','No se pudo cargar la lista'); throw e
      } finally {
        commit('setLoading', false)
      }
    }
}}
```

Usos clásicos: Utilizando Vuex y Axios

Consumir desde el componente

La vista **despacha** la action en el montaje y **refleja** loading/error/items que vienen del store. Nada de pegar then/catch en la vista: eso quedó dentro de la action.

```vue
<!-- FrameworksView.vue -->
<script setup>
import { computed, onMounted } from 'vue'
import { useStore } from 'vuex'

const store = useStore()
const frameworks = computed(() => store.state.frameworks.items)
const loading    = computed(() => store.state.frameworks.loading)
const error      = computed(() => store.state.frameworks.error)

onMounted(() => store.dispatch('frameworks/cargar'))
</script>

<template>
  <section>
    <p v-if="loading">Cargando...</p>
    <p v-else-if="error">{{ error }}</p>
    <ul v-else>
      <li v-for="f in frameworks" :key="f.id">{{ f.nombre }}</li>
    </ul>
  </section>
</template>
```

Bootcamp
Academy
sustantiva